

Motivating the BFGS Method

From Newton's Method to Quasi-Newton Updates

Robert Hildebrand

Industrial and Systems Engineering
Virginia Tech

Reading: Nocedal & Wright, Chapter 6

Newton's Method: The Gold Standard

The update:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$$

Minimizes the **quadratic model** of f at $\mathbf{x}_k$:

$$m_k(\mathbf{p}) = f_k + \nabla f_k^\top \mathbf{p} + \frac{1}{2} \mathbf{p}^\top \nabla^2 f_k \mathbf{p}$$

Why it's great:

- ✓ **Quadratic convergence** near $\mathbf{x}^*$
- ✓ Uses **curvature information** (the Hessian)
- ✓ **Scale-invariant**: not degraded by poor conditioning like steepest descent
- ✓ Converges in **1 step** for quadratic objectives

The Cost of Newton's Method

- ✗ **Computing the Hessian** $\nabla^2 f(\mathbf{x}_k)$

Requires $n(n+1)/2$ second derivatives at every iteration.

For $n = 1,000$, that's $\sim 500,000$ entries.

- ✗ **Solving the linear system** $\nabla^2 f(\mathbf{x}_k) \mathbf{p}_k = -\nabla f(\mathbf{x}_k)$

$O(n^3)$ cost per iteration via direct factorization.

- ✗ **The Hessian may not be available analytically**

Many practical problems only provide f and ∇f (e.g., simulation-based optimization, ML training).

Can we get Newton-like convergence without computing the Hessian?

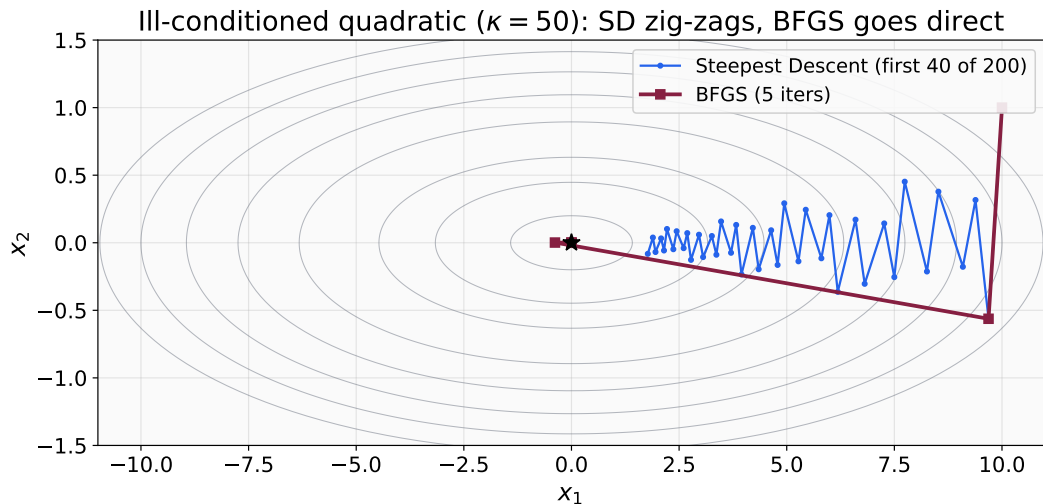
The Tradeoff: Cost vs. Convergence

	Steepest Descent	Quasi-Newton	Newton
Cost/iteration	$O(n)$	$O(n^2)$	$O(n^3)$
Convergence	Linear	Superlinear	Quadratic
Requires	∇f	∇f	$\nabla f, \nabla^2 f$
Storage	$O(n)$	$O(n^2)$	$O(n^2)$
✓ Pro	Cheap	Fast, no Hessian	Fastest convergence
✗ Con	Slow, zig-zags	$O(n^2)$ storage	Expensive per step

Steepest Descent $\longrightarrow$ Quasi-Newton $\longrightarrow$ Newton

Increasing cost per iteration, faster convergence

Why Curvature Matters: SD vs. BFGS



The Quasi-Newton Idea

Key Idea

Replace $\nabla^2 f(\mathbf{x}_k)$ with an approximation $\mathbf{B}_k$ that we **update cheaply** using gradient information we already compute.

Newton: $\mathbf{B}_k \mathbf{p}_k = -\nabla f_k$ where $\mathbf{B}_k = \nabla^2 f(\mathbf{x}_k)$

Quasi-Newton: $\mathbf{B}_k \mathbf{p}_k = -\nabla f_k$ where $\mathbf{B}_k \approx \nabla^2 f(\mathbf{x}_k)$

The central question: How should we choose and update $\mathbf{B}_k$ so that:

- 1 It captures useful **curvature information**
- 2 The update from $\mathbf{B}_k$ to $\mathbf{B}_{k+1}$ is **cheap** (rank-1 or rank-2)
- 3 $\mathbf{B}_k$ remains **symmetric positive definite**
- 4 We still get **fast (superlinear) convergence**

The Secant Condition

From Taylor's theorem applied to ∇f , for the true Hessian:

$$\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \approx \nabla^2 f(\mathbf{x}_{k+1}) (\mathbf{x}_{k+1} - \mathbf{x}_k)$$

Define the key quantities:

$$\boxed{\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k} \quad (\text{step}) \qquad \boxed{\mathbf{y}_k = \nabla f_{k+1} - \nabla f_k} \quad (\text{gradient change})$$

We require that $\mathbf{B}_{k+1}$ satisfies the **secant equation**:

$$\boxed{\mathbf{B}_{k+1} \mathbf{s}_k = \mathbf{y}_k}$$

Interpretation: $\mathbf{B}_{k+1}$ acts like the true Hessian along the most recent step direction.

The Secant Condition Isn't Enough

The secant equation $\mathbf{B}_{k+1}\mathbf{s}_k = \mathbf{y}_k$ gives n equations for an $n \times n$ symmetric matrix with $\frac{n(n+1)}{2}$ unknowns.

This system is **massively underdetermined**. We need additional conditions:

❶ **Symmetry:** $\mathbf{B}_{k+1} = \mathbf{B}_{k+1}^\top$

The true Hessian is symmetric, so our approximation should be too.

❷ **Positive definiteness:** $\mathbf{B}_{k+1} \succ 0$

Ensures the search direction $\mathbf{p}_k = -\mathbf{B}_k^{-1}\nabla f_k$ is a descent direction.

❸ **Minimal change from $\mathbf{B}_k$:**

We've already built up curvature information in $\mathbf{B}_k$ — don't throw it away.

BFGS: Closest Matrix Satisfying the Secant Condition

The BFGS update solves:

$$\begin{array}{ll} \min_{\mathbf{B}} & \|\mathbf{B} - \mathbf{B}_k\|_W \\ \text{s.t.} & \mathbf{B} = \mathbf{B}^\top, \quad \mathbf{B} \mathbf{s}_k = \mathbf{y}_k \end{array}$$

where $\|\cdot\|_W$ is the weighted Frobenius norm:

$$\|A\|_W = \|W^{1/2} A W^{1/2}\|_F$$

with $W = \bar{G}_k^{-1}$, where $\bar{G}_k$ is any matrix satisfying $\bar{G}_k \mathbf{s}_k = \mathbf{y}_k$.

Intuition

Among all symmetric matrices that match the most recent curvature data (secant condition), pick the one that **changes $\mathbf{B}_k$ as little as possible**. This preserves previously accumulated curvature information.

Why a Weighted Norm?

Why not just use the ordinary Frobenius norm $\|\mathbf{B} - \mathbf{B}_k\|_F$?

Consider a change of variables $\mathbf{x} \rightarrow A\mathbf{x}$ for some invertible A .

The Hessian transforms as $\mathbf{B} \rightarrow A^{-\top} \mathbf{B} A^{-1}$.

The Frobenius norm is **not** invariant under this transformation:

$$\|A^{-\top} \mathbf{B} A^{-1} - A^{-\top} \mathbf{B}_k A^{-1}\|_F \neq \|\mathbf{B} - \mathbf{B}_k\|_F \quad \text{in general.}$$

So the “closest” matrix **depends on your choice of coordinates** — clearly undesirable.

What we want: A norm that measures closeness in a way that respects the **intrinsic geometry** of the problem, not the arbitrary choice of units/coordinates.

Natural idea: Measure distances in a coordinate system adapted to the *curvature* of f — i.e., weight by something related to the Hessian.

The Choice of W : Not as Arbitrary as It Looks

We require $W\mathbf{y}_k = \mathbf{s}_k$, i.e., W acts like an **inverse average Hessian** along the step (since $\mathbf{y}_k \approx \bar{G}_k \mathbf{s}_k$ for some average Hessian $\bar{G}_k$, so $W = \bar{G}_k^{-1}$).

Why this condition?

- It makes the weighted norm **scale-invariant**: applying $\mathbf{x} \rightarrow A\mathbf{x}$ transforms the problem consistently.
- It gives W the right “units” — $W^{1/2} \mathbf{B} W^{1/2}$ is dimensionless.

Uniqueness

Every $W \succ 0$ satisfying $W\mathbf{y}_k = \mathbf{s}_k$ gives the **same BFGS update**. The solution is unique despite the apparent freedom in choosing W .

The “arbitrariness” is an illusion: $W\mathbf{y}_k = \mathbf{s}_k$ is the minimal condition for scale invariance, and once imposed, the answer doesn’t depend on which W you pick.

Setting Up the Derivation

We derive the BFGS update directly from the $\mathbf{B}$ formulation:

$$\begin{array}{ll} \min_{\mathbf{B}} & \|W^{1/2}(\mathbf{B} - \mathbf{B}_k)W^{1/2}\|_F^2 \\ \text{s.t.} & \mathbf{B} = \mathbf{B}^\top, \quad \mathbf{B} \mathbf{s}_k = \mathbf{y}_k \end{array}$$

where $W^{-1}\mathbf{y}_k = \mathbf{s}_k$ (i.e., W^{-1} acts like an average Hessian).

The constraint $\mathbf{B}\mathbf{s}_k = \mathbf{y}_k$ is a system of n **linear** equations in the entries of $\mathbf{B}$.

⇒ This is a **linearly constrained convex optimization problem** with a quadratic objective.

Key simplification: Symmetry of $\mathbf{B}$ is *not* a separate constraint — if $\mathbf{B}_k$ is symmetric, the optimal solution is automatically symmetric (the weighted Frobenius norm and secant constraint both respect symmetry).

The Frobenius Norm

Frobenius norm: For an $n \times n$ matrix A ,

$$\|A\|_F^2 = \sum_{i,j} A_{ij}^2 = \text{tr}(A^\top A)$$

The trace form follows because $\text{tr}(A^\top A) = \sum_i (A^\top A)_{ii} = \sum_i \sum_j A_{ji} A_{ji} = \sum_{i,j} A_{ij}^2$.

Weighted Frobenius norm: For $W \succ 0$ and symmetric A ,

$$\|A\|_W^2 = \|W^{1/2} A W^{1/2}\|_F^2 = \text{tr}((W^{1/2} A W^{1/2})^\top (W^{1/2} A W^{1/2}))$$

Since $W^\top = W$ and $A^\top = A$:

$$= \text{tr}(W^{1/2} A W^{1/2} W^{1/2} A W^{1/2}) = \text{tr}(W A W A)$$

So our objective is $\phi(\mathbf{B}) = \text{tr}(W(\mathbf{B} - \mathbf{B}_k)W(\mathbf{B} - \mathbf{B}_k))$.

We need $\frac{\partial \phi}{\partial \mathbf{B}}$.

Differentiating $\text{tr}(WDWD)$ with Respect to D

Let $D = \mathbf{B} - \mathbf{B}_k$ so $\phi = \text{tr}(WDWD)$. We compute $\frac{\partial \phi}{\partial D}$.

Step 1. For any matrices P, Q of compatible size, $\frac{\partial}{\partial X_{ij}} \text{tr}(PXQ) = (QP)_{ji}$, so $\nabla_X \text{tr}(PXQ) = P^\top Q^\top$.

Step 2. Since D appears *twice* in $\text{tr}(WDWD)$, apply the product rule:

$$\frac{\partial}{\partial D} \text{tr}(W \underbrace{D}_{(1)} W \underbrace{D}_{(2)}) = \underbrace{\nabla_D \text{tr}((WDW) \cdot D)}_{\text{vary 2nd } D} + \underbrace{\nabla_D \text{tr}(W \cdot D \cdot (WD))}_{\text{vary 1st } D}$$

Applying Step 1 to each term (with $P^\top Q^\top$ and using $W^\top = W$, $D^\top = D$):

$$\text{Vary 2nd } D : \quad P = WDW, \quad Q = I \quad \Rightarrow \quad (WDW)^\top = WDW$$

$$\text{Vary 1st } D : \quad P = W, \quad Q = WD \quad \Rightarrow \quad W(WD)^\top = WDW$$

$$\frac{\partial \phi}{\partial D} = WDW + WDW = 2W(\mathbf{B} - \mathbf{B}_k)W$$

KKT Conditions for Symmetric $\mathbf{B}$

Introduce Lagrange multiplier $\boldsymbol{\lambda} \in \mathbb{R}^n$ for the constraint $\mathbf{B}\mathbf{s}_k = \mathbf{y}_k$.

The Lagrangian:

$$L(\mathbf{B}, \boldsymbol{\lambda}) = \underbrace{\text{tr}(W(\mathbf{B} - \mathbf{B}_k)W(\mathbf{B} - \mathbf{B}_k))}_{\|W^{1/2}(\mathbf{B} - \mathbf{B}_k)W^{1/2}\|_F^2} + \boldsymbol{\lambda}^\top (\mathbf{B}\mathbf{s}_k - \mathbf{y}_k)$$

Stationarity. From the previous slide, $\frac{\partial}{\partial \mathbf{B}} \text{tr}(W(\mathbf{B} - \mathbf{B}_k)W(\mathbf{B} - \mathbf{B}_k)) = 2W(\mathbf{B} - \mathbf{B}_k)W$.

For the constraint term $\boldsymbol{\lambda}^\top \mathbf{B}\mathbf{s}_k$, note $\frac{\partial}{\partial \mathbf{B}} \boldsymbol{\lambda}^\top \mathbf{B}\mathbf{s}_k = \boldsymbol{\lambda} \mathbf{s}_k^\top$. Since $\mathbf{B}$ is **symmetric**, we symmetrize the constraint gradient:

$$2W(\mathbf{B} - \mathbf{B}_k)W + \frac{1}{2}(\boldsymbol{\lambda} \mathbf{s}_k^\top + \mathbf{s}_k \boldsymbol{\lambda}^\top) = 0$$

(When optimizing $f(\mathbf{B})$ over symmetric matrices, $\nabla_{\mathbf{B}} g = \frac{1}{2}(\boldsymbol{\lambda} \mathbf{s}_k^\top + \mathbf{s}_k \boldsymbol{\lambda}^\top)$ rather than $\boldsymbol{\lambda} \mathbf{s}_k^\top$, because perturbing a symmetric matrix $\mathbf{B} \rightarrow \mathbf{B} + \Delta$ requires $\Delta = \Delta^\top$.)

Solving for the correction:

$$\mathbf{B}_{k+1} - \mathbf{B}_k = -\frac{1}{4}W^{-1}(\boldsymbol{\lambda} \mathbf{s}_k^\top + \mathbf{s}_k \boldsymbol{\lambda}^\top)W^{-1}$$

Determining λ from the Secant Condition

Substitute $\mathbf{B}_{k+1} = \mathbf{B}_k - \frac{1}{4}W^{-1}(\lambda \mathbf{s}_k^\top + \mathbf{s}_k \lambda^\top)W^{-1}$ into $\mathbf{B}_{k+1}\mathbf{s}_k = \mathbf{y}_k$:

$$\mathbf{B}_k \mathbf{s}_k - \frac{1}{4} \left[W^{-1} \lambda \underbrace{(\mathbf{s}_k^\top W^{-1} \mathbf{s}_k)}_{\gamma} + W^{-1} \mathbf{s}_k \underbrace{(\lambda^\top W^{-1} \mathbf{s}_k)}_{\delta} \right] = \mathbf{y}_k$$

Using $W^{-1}\mathbf{s}_k = \mathbf{y}_k$ (from $W\mathbf{y}_k = \mathbf{s}_k$):

$$\frac{\gamma}{4} W^{-1} \lambda + \frac{\delta}{4} \mathbf{y}_k = \mathbf{B}_k \mathbf{s}_k - \mathbf{y}_k$$

Take inner product with $\mathbf{s}_k$ (noting $\mathbf{s}_k^\top W^{-1} \lambda = \delta$ and $\mathbf{s}_k^\top \mathbf{y}_k = \mathbf{y}_k^\top \mathbf{s}_k$):

$$\frac{\gamma\delta}{4} + \frac{\delta}{4}(\mathbf{y}_k^\top \mathbf{s}_k) = \mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k - \mathbf{y}_k^\top \mathbf{s}_k$$

This determines δ , then $W^{-1}\lambda$, and finally the full correction.

Completing the KKT Derivation

After solving for λ and substituting back, the correction simplifies to:

$$\mathbf{B}_{k+1} - \mathbf{B}_k = -\frac{1}{4}W^{-1}(\lambda \mathbf{s}_k^\top + \mathbf{s}_k \lambda^\top)W^{-1}$$

Using $W^{-1}\mathbf{s}_k = \mathbf{y}_k$ throughout, this reduces to:

$$\boxed{\mathbf{B}_{k+1} = \mathbf{B}_k - \frac{\mathbf{B}_k \mathbf{s}_k \mathbf{s}_k^\top \mathbf{B}_k}{\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k} + \frac{\mathbf{y}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k}}$$

The algebra is manageable but not especially illuminating. The **rank-2 ansatz** on the next slide provides a more direct path to the same result.

A Cleaner Derivation: The Rank-2 Ansatz

Alternative approach: Seek $\mathbf{B}_{k+1} = \mathbf{B}_k + \alpha \mathbf{u} \mathbf{u}^\top + \beta \mathbf{v} \mathbf{v}^\top$ (symmetric rank-2 update).

The secant condition $\mathbf{B}_{k+1} \mathbf{s}_k = \mathbf{y}_k$ gives:

$$\mathbf{B}_k \mathbf{s}_k + \alpha (\mathbf{u}^\top \mathbf{s}_k) \mathbf{u} + \beta (\mathbf{v}^\top \mathbf{s}_k) \mathbf{v} = \mathbf{y}_k$$

Choose $\mathbf{u} = \mathbf{y}_k$ and $\mathbf{v} = \mathbf{B}_k \mathbf{s}_k$. Then:

$$\mathbf{B}_k \mathbf{s}_k + \alpha (\mathbf{y}_k^\top \mathbf{s}_k) \mathbf{y}_k + \beta (\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k) \mathbf{B}_k \mathbf{s}_k = \mathbf{y}_k$$

Match coefficients of $\mathbf{y}_k$ and $\mathbf{B}_k \mathbf{s}_k$:

$$\text{coeff. of } \mathbf{y}_k : \quad \alpha (\mathbf{y}_k^\top \mathbf{s}_k) = 1 \quad \implies \quad \alpha = \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k}$$

$$\text{coeff. of } \mathbf{B}_k \mathbf{s}_k : \quad 1 + \beta (\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k) = 0 \quad \implies \quad \beta = \frac{-1}{\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k}$$

$$\boxed{\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{\mathbf{y}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} - \frac{\mathbf{B}_k \mathbf{s}_k \mathbf{s}_k^\top \mathbf{B}_k}{\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k}}$$

Why Rank 2? — Summary

The BFGS correction $\mathbf{B}_{k+1} - \mathbf{B}_k$ is:

$$\mathbf{B}_{k+1} - \mathbf{B}_k = \underbrace{\frac{\mathbf{y}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k}}_{\text{rank-1: adds new curvature from } \mathbf{y}_k} - \underbrace{\frac{\mathbf{B}_k \mathbf{s}_k \mathbf{s}_k^\top \mathbf{B}_k}{\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k}}_{\text{rank-1: removes old info along } \mathbf{s}_k}$$

The correction lives in $\text{span}\{\mathbf{y}_k, \mathbf{B}_k \mathbf{s}_k\}$.

Interpretation:

- The $-\mathbf{B}_k \mathbf{s}_k \mathbf{s}_k^\top \mathbf{B}_k / (\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k)$ term **removes** $\mathbf{B}_k$'s action along $\mathbf{s}_k$
(check: this term applied to $\mathbf{s}_k$ gives $-\mathbf{B}_k \mathbf{s}_k$, canceling $\mathbf{B}_k \mathbf{s}_k$)
- The $\mathbf{y}_k \mathbf{y}_k^\top / (\mathbf{y}_k^\top \mathbf{s}_k)$ term **installs** the correct curvature
(check: this term applied to $\mathbf{s}_k$ gives $\mathbf{y}_k$, which is what we need)
- Together: $\mathbf{B}_{k+1} \mathbf{s}_k = \mathbf{B}_k \mathbf{s}_k - \mathbf{B}_k \mathbf{s}_k + \mathbf{y}_k = \mathbf{y}_k \quad \checkmark$

From the **B** Update to the **H** Update

We want $\mathbf{H}_{k+1} = \mathbf{B}_{k+1}^{-1}$ for efficient computation of $\mathbf{p}_k = -\mathbf{H}_k \nabla f_k$.

Apply the **Sherman–Morrison–Woodbury** formula to $\mathbf{B}_{k+1} = \mathbf{B}_k + \mathbf{a}\mathbf{b}^\top + \mathbf{c}\mathbf{d}^\top$ as a rank-2 update, using $U = [\mathbf{a} \ \mathbf{c}]$, $V^\top = [\mathbf{b} \ \mathbf{d}]^\top$.

After algebra (or by direct verification that $\mathbf{B}_{k+1}\mathbf{H}_{k+1} = \mathbf{I}$):

$$\mathbf{H}_{k+1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) \mathbf{H}_k (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$$

where $\rho_k = 1/(\mathbf{y}_k^\top \mathbf{s}_k)$.

Key observations:

- Also a **rank-2 update** — $O(n^2)$ work, no linear system solve
- Using $\mathbf{H}_k$ directly: $\mathbf{p}_k = -\mathbf{H}_k \nabla f_k$ is just a matrix-vector product
- Only requires ∇f — no second derivatives needed

BFGS vs. DFP: A Duality

What if we minimize over $\mathbf{H}$ instead of $\mathbf{B}$?

	BFGS	DFP
Minimizes	$\ \mathbf{B} - \mathbf{B}_k\ _W$	$\ \mathbf{H} - \mathbf{H}_k\ _{\hat{W}}$
Subject to	$\mathbf{B}\mathbf{s}_k = \mathbf{y}_k$	$\mathbf{H}\mathbf{y}_k = \mathbf{s}_k$
Directly controls	Hessian approx. $\mathbf{B}$	Inverse Hessian $\mathbf{H}$

The DFP formula for $\mathbf{H}$ is *exactly* the BFGS formula for $\mathbf{B}$ with $\mathbf{s}_k \leftrightarrow \mathbf{y}_k$:

$$\text{DFP: } \mathbf{H}_{k+1} = \mathbf{H}_k - \frac{\mathbf{H}_k \mathbf{y}_k \mathbf{y}_k^\top \mathbf{H}_k}{\mathbf{y}_k^\top \mathbf{H}_k \mathbf{y}_k} + \frac{\mathbf{s}_k \mathbf{s}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k}$$

Why BFGS wins in practice: Errors in $\mathbf{B}$ are “smoothed” when computing $\mathbf{p} = -\mathbf{B}^{-1}\nabla f$. Errors in $\mathbf{H}$ (as in DFP) translate *directly* into bad search directions. BFGS is significantly more **self-correcting** with inexact line searches.

Why BFGS Works So Well

Positive definiteness preserved

If $\mathbf{B}_k \succ 0$ and $\mathbf{y}_k^\top \mathbf{s}_k > 0$ (guaranteed by Wolfe conditions), then $\mathbf{B}_{k+1} \succ 0$.

Descent direction at every step.

Self-correcting

Bad curvature estimates get corrected in subsequent steps. BFGS is remarkably robust to poor initial choices of $\mathbf{B}_0$ (typically $\mathbf{B}_0 = \mathbf{I}$).

Superlinear convergence

Dennis–Moré condition: $\mathbf{B}_k \rightarrow \nabla^2 f(\mathbf{x}^*)$ along the search directions.

Convergence is superlinear without ever computing the true Hessian.

Finite termination on quadratics

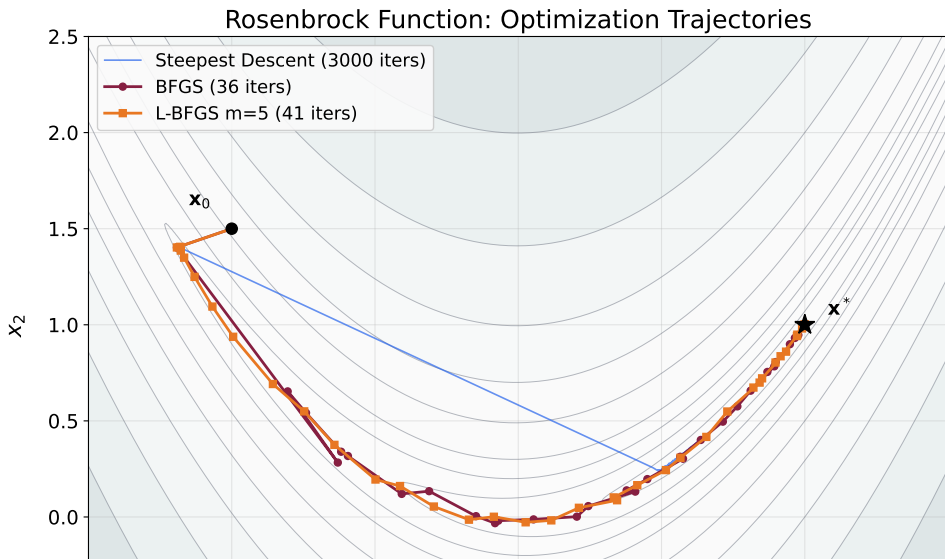
On $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x}$, BFGS with exact line search converges in at most n steps, and $\mathbf{B}_n = \mathbf{A}$ exactly.

BFGS Algorithm

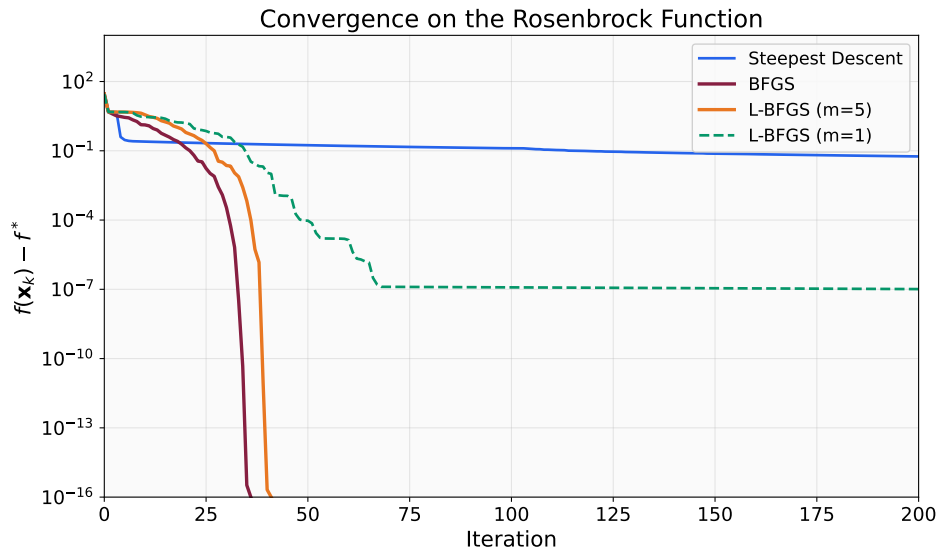
- 0 Choose $\mathbf{x}_0$, set $\mathbf{H}_0 = \mathbf{I}$ (or other SPD matrix), $k = 0$
- 1 Compute search direction: $\mathbf{p}_k = -\mathbf{H}_k \nabla f(\mathbf{x}_k)$
- 2 Line search: find α_k satisfying the **Wolfe conditions**
- 3 Set $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$
- 4 Compute $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$, $\mathbf{y}_k = \nabla f_{k+1} - \nabla f_k$, $\rho_k = 1/(\mathbf{y}_k^\top \mathbf{s}_k)$
- 5 Update: $\mathbf{H}_{k+1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) \mathbf{H}_k (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$
- 6 Set $k = k + 1$, go to Step 1

Cost per iteration: $O(n^2)$ — one matrix-vector product + rank-2 update.

BFGS in Action: Rosenbrock Function



Convergence Comparison



Looking Ahead: L-BFGS

BFGS stores a dense $n \times n$ matrix — not feasible when n is very large (e.g., deep learning with millions of parameters).

Limited-memory BFGS (L-BFGS):

- Store only the last m pairs $\{\mathbf{s}_k, \mathbf{y}_k\}$ (typically $m = 3\text{--}20$)
- Compute $\mathbf{H}_k \nabla f_k$ implicitly via the **two-loop recursion**
- $O(mn)$ storage and $O(mn)$ cost per iteration
- The workhorse for **large-scale smooth optimization**

Next time: L-BFGS two-loop recursion and practical implementation.

Case Study: L-BFGS in Physics-Informed Neural Networks

Physics-Informed Neural Networks (PINNs) solve PDEs by training a neural network $u_\theta(\mathbf{x}, t)$ to satisfy:

$$\mathcal{L}(\theta) = \underbrace{\lambda_{\text{pde}} \sum_i |\mathcal{N}[u_\theta](\mathbf{x}_i, t_i)|^2}_{\text{PDE residual}} + \underbrace{\lambda_{\text{bc}} \sum_j |u_\theta(\mathbf{x}_j, t_j) - g_j|^2}_{\text{boundary/initial conditions}}$$

where $\mathcal{N}$ is a differential operator (e.g., $u_t + u u_x - \nu u_{xx} = 0$ for Burgers' equation).

Key features:

- **Mesh-free:** no spatial discretization needed
- **Gradients via automatic differentiation:** $\nabla_\theta \mathcal{L}$ computed exactly
- The loss $\mathcal{L}$ is a **smooth, deterministic** function of $\theta \Rightarrow$ ideal for L-BFGS

Why L-BFGS Excels for PINNs

Standard practice: Two-stage training (Raissi et al., 2019 and virtually all subsequent work):

- ➊ **Phase 1: Adam** (first-order, stochastic) — $\sim 10,000$ iterations

Escapes bad local regions, handles noisy early gradients, cheap per step.

- ➋ **Phase 2: L-BFGS** (quasi-Newton, deterministic) — until convergence

Exploits curvature to rapidly converge. Typically reduces loss by **2–4 orders of magnitude** beyond what Adam achieves.

Why does L-BFGS work so well here?

- PINN losses are **smooth** (neural net + automatic differentiation of PDE)
- Full-batch training is common (all collocation points used each step) $\Rightarrow$ **deterministic gradients**, so quasi-Newton curvature estimates are accurate
- Typical network sizes: $n \sim 10^3\text{--}10^4$ parameters $\Rightarrow$ L-BFGS with $m = 20$ is very efficient

Example: Solving Burgers' Equation with a PINN

Burgers' equation: $u_t + u u_x = \nu u_{xx}$, $x \in [-1, 1]$, $t \in [0, 1]$, $\nu = 0.01/\pi$

Network: 4 hidden layers $\times$ 20 neurons, tanh activations ($n \approx 1,300$ parameters).

Adam only (50k iterations):

- Loss $\sim 10^{-3}$
- Relative L^2 error $\sim 10^{-2}$
- Smooth features captured
- Struggles with sharp gradients

Adam + L-BFGS:

- Loss $\sim 10^{-6}$ – 10^{-7}
- Relative L^2 error $\sim 10^{-4}$
- Sharp shock front resolved
- Converges in ~ 1000 L-BFGS iterations

The lesson

L-BFGS provides the “last mile” of convergence that first-order methods cannot, precisely because it builds up curvature information via the BFGS update — the theory we just developed.

Why Not Just Use Adam?

The loss landscape near a PDE solution is smooth and well-conditioned.

This is exactly where quasi-Newton methods shine and SGD-type methods struggle:

	Adam	L-BFGS
Per-iteration cost	$O(n)$	$O(mn)$, $m \ll n$
Convergence near minimum	Linear (at best)	Superlinear
Stochastic gradients?	Yes (designed for it)	No (needs full batch)
Hyperparameters	Learning rate, β_1, β_2	Almost none
Curvature info	Diagonal approx. only	Full directional curvature

Near a solution, L-BFGS's Hessian approximation $\mathbf{H}_k$ captures the **anisotropy** of the loss landscape — some directions curve steeply, others are nearly flat. Adam treats all directions roughly equally.

References: Raissi, Perdikaris, Karniadakis (JCP, 2019); Cuomo et al. (J. Sci. Comput., 2022).